

§ Le service logger §

PRÉREQUIS

Avant de pouvoir commencer la séance, vous devez :

1. avoir lu les présentations 4.1 et 4.2 du cours ;
2. avoir fait la séance 3 ou récupérer les solutions.

OBJECTIFS DE LA SÉANCE

1. Comprendre l'architecture en services d'un serveur.
2. Pouvoir modifier et utiliser un logger simple.
3. Mettre une information de logging sur toutes les routes.

INTRODUCTION AUX SERVICES

Nous l'avons vu dans les présentations, l'architecture retenue pour notre serveur se compose de **contrôleurs**, de **modèles** et de **services**. Les services sont des classes qui contiennent les fonctions qui permettent (le plus souvent, mais pas que) de manipuler les données. Les contrôleurs utilisent ces services pour effectuer les opérations demandées par les routes.

Cette architecture permet de séparer les différentes responsabilités de notre serveur. De plus, lorsqu'on veut changer une partie de notre technologie (par exemple, changer de base de données, ou de logger), il suffit de modifier le service correspondant et on peut laisser le reste inchangé.

L'architecture actuelle n'utilise pas encore de service. Aucun de vos contrôleurs n'utilise de services. C'est l'objectif de cette séance que de les introduire.

GESTION DES ERREURS : LOGGER SIMPLE

Tout d'abord, créez un dossier **logs** à la racine de votre projet. Ce dossier doit se trouver au même niveau que les dossiers **src** et **http**.

Observez le fichiers `services/logger.service.ts`. Ce fichier contient une classe `LoggerService` qui permet de gérer les logs de notre serveur. Les logs sont un élément important de toute infrastructure informatique. Ils permettent d'assurer une traçabilité des actions effectuées par le serveur. Ils permettent également de déboguer le serveur en cas de problème.

Un logger se découpe en plusieurs niveaux de log. Dans notre cas, nous utiliserons :

- `info` : pour les informations générales utiles à l'administrateur du serveur.
- `error` : pour les erreurs permettant ainsi de retrouver celles-ci lorsqu'elles se produisent.
- `debug` : pour les informations de débogage liée au développement du serveur.

Ainsi, une information qui n'est intéressante que durant le développement peut être loggée en `debug`, alors qu'une erreur doit être loggée sur le canal `error`.

EXERCICE 1: MISE EN PLACE DU LOGGER SUR TOUTES LES ROUTES

À partir de maintenant, vous devez ajouter - au moins - une ligne de log sur chaque route de votre serveur. Cette ligne doit contenir le nom de la route, la méthode HTTP utilisée et le ou les paramètres de la route si elle en possède.

Reprenez tous les contrôleurs de votre serveur. Pour chaque route, la première ligne de la fonction doit être commande écrivant sur le canal `info` du logger. Par exemple

```
LoggerService.info('GET /doctors')
```

Ici la route appelée ne contient aucun paramètre.

Si une route échoue à effectuer ce pour quoi on l'appelle, le programme doit écrire un log sur le canal `error` avant d'envoyer toute réponse au client.

Cet exercice vous demande de repasser sur toutes vos routes et de les modifier pour ajouter ces logs. Vous pouvez également rajouter, sur le canal `debug`, tout log que vous jugerez utile.

A termes, plus aucun `console.log()` ne doit être présent dans votre code (excepté dans le `LoggerService`).

Effectuez un test de toutes vos routes pour vérifier que les logs sont bien écrits. Lorsqu'on écrit un log sur le canal `error`, un fichier est créé. Ce fichier sera créé dans un dossier `logs` à la racine de votre projet. Vous devez créer ce dossier si il n'existe pas.

EXERCICE 2: GET AVEC FILTRAGE

PRÉPARATION DES DONNÉES

Reprenez le fichier `patient.model.ts` et ajoutez une nouvelle interface `PatientFilter` qui contiendra les critères de filtrage des patients. Cette interface contiendra la propriété suivante :

- `zipCode` : string;

Cette propriété est *optionnelle*.

FILTRAGE SUR L'ADRESSE DES PATIENTS

A termes, il n'est pas possible d'avoir une route pour obtenir les patients selon certains critères (par exemple par localité, le docteur de référence, etc.). Il est bien plus simple d'effectuer

du filtrage sur la route `GET /patients`.

Nous allons reprendre la route `GET /patients` de la fiche 2.1 et nous allons y introduire un filtrage sur l'adresse des patients. Pour cela, nous allons utiliser la query de l'url et la clé `zipCode`. Si cette query est présente, la route ne devra renvoyer que les patients qui ont une adresse avec ce code postal. Si la query n'est pas présente, la route devra renvoyer tous les patients.

Pour implémenter ce filtrage, nous allons :

- Modifier le fichier `patients.http` pour ajouter une nouvelle requête avec une query contenant une clé `zipCode` et une valeur (correspondant à un code postal existant dans les données).
- Modifier le contrôleur et extraire le code postal de la query de la requête.
- Vérifier la présence de la query `zipCode` à la fonction de *guard* adaptée.
- Si la query est présente, nous allons créer un tableau vide et, à l'aide d'une boucle `for`, parcourir tous les patients et ajouter ceux qui ont le code postal demandé dans le tableau.